Flake – Final Team Presentation

- A Collaborative Trip Planning Application
- Team 4 - Mountain Hackers
  - Aaron Brunswig
  - Yan Gu
  - Spencer Iascone
  - Joshua Morrison
  - London Walker
- Course: CSPB 3308 Software Development Tools & Methods, Spring 2026

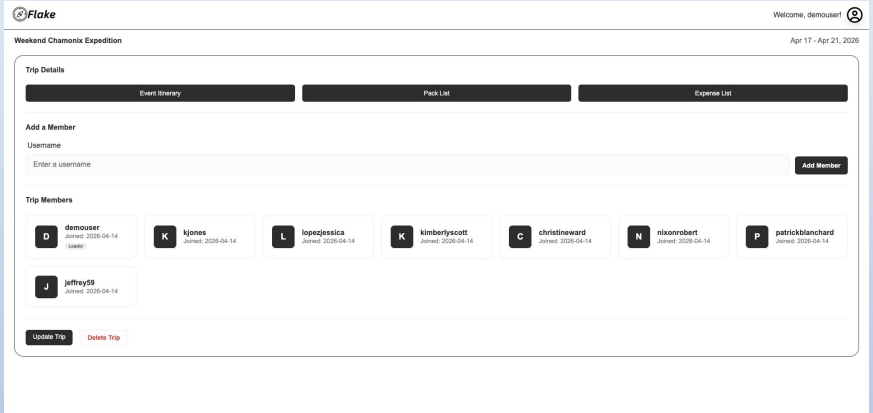
Problem Motivation

- The Problem: Group trips often suffer from “Diffusion of Responsibility”:
  - Everyone assumes someone else has it covered (eg. essential items to carry) until it’s too late to be found not covered
  - Everyone assumes someone else is tracking itemized expense for accurate cost split calculation until it’s too late and can only be made by rough estimation
- The Motivation: Collective Accountability
  - Claimed Ownership: trip members identify what they are responsible for
  - Financial Transparency: track itemized expenses incurred during the trip to be shared among the group members
  - Proactive Recovery: redistribution of responsibilities when a trip member quits or simply “flakes”

What Flake Does

Flake enables groups to:

- Create shared trip logistics
- Establish ownership of logistics
- Visualize group readiness
- Allow users to reassign responsibilities when a team member flakes
- Centralized organization to eliminate “Diffusion of Responsibility” problem



Trip overview page displaying links to events, items and expenses as well as member list and member add form

System Architecture Overview

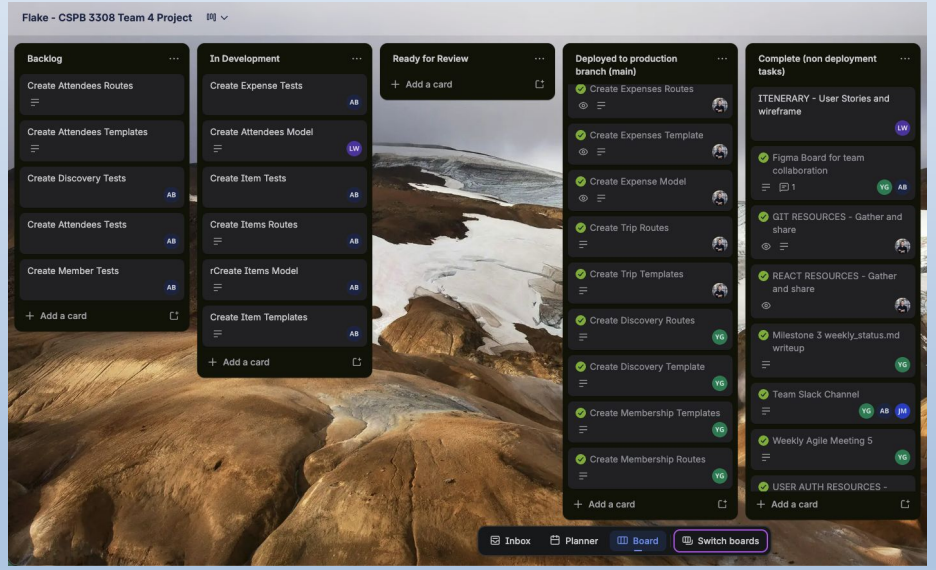
- **Frontend:** Flask rendered Jinja2 Templates (server-side rendering)
- **Backend:** Flask / Python3
- **Database:** SQLAlchemy / SQLite3 (Local development instance)
- **Version Control:** Git / GitHub (Team Collaboration and branch management)
- **Local Environment:** Manual local deployment (Flask development server)

Key Features

Implemented

- User Authentication and Account Creation
- Trip Creation and Management
- Event Creation and Management
- Expense Creation and Management
- Public Discovery Page
- Trip Memberships
- Pack List / Item management

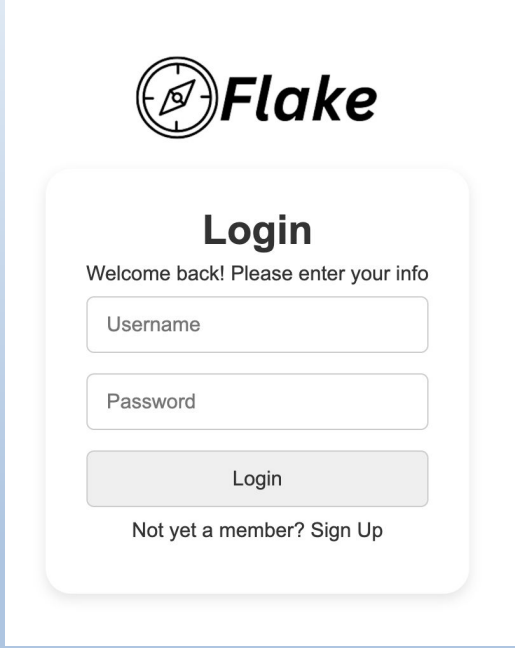
Team Trello board



Demo Flow Overview

Our live demo will follow this sequence:

1. User Signup and Login
2. Creating a new Trip
3. Trip Membership
   - Add a member to the trip
   - Leave a trip
4. Adding/Editing an Event
5. Adding/Editing an Item
6. Adding/Editing an Expense
7. Discovering public trips



Flake Login Page

Flake Demonstration

Major Technical Challenges

- Flask Routing and Navigation
  - Managing URL patterns and resolving function name collisions
- Object Relational Mapping (ORM):
  - Syncing Python classes with SQLite database via SQLAlchemy
- Dynamic Jinja2 Templating
  - Passing complex data structures from the backend to the frontend
- Testing Implementation:
  - Integrating an effective test framework and establishing a base set of tests for each feature

What Went Well

- Effective Team Communication
  - Moved from email to slack to facilitate instant communication
  - Consistent use of slack to troubleshoot any technical issues and share status updates
- Fast PR review cycle
  - PRs typically resolved within 24-48 hours
- Clear division of Responsibilities
  - Each team member owns a specific feature
  - Trello to track tasks self-assigned
  - Take turns to play the scrum master role
- Rapid Early Prototyping
  - Initial iterations of user authorization led to efficient streamlining of project scope and architecture

What Didn’t Go Well

- Tech Stack Switch
  - Initial designs called for tools that were outside of course materials and the team’s time availability
    - Made quick team decision to reassign React as a STRETCH goal
    - Immediately sped up development for V1 of Flake
- Learning Curve Hurdles
  - Slow progression in the beginning while we spent time mastering Flask, Jinja2 and SQL from lab assignments
- Environment Configuration
  - Struggles while we settled on our base backend configuration
    - Side effect of learning a new framework

Lessons Learned

- Iteration over Perfection:
  - Getting the basics working before adding complex functionality saves time
- Document Everything
  - Outlining local setup instructions helps reduce friction when integrating newly merged features
- Mastering Routing
  - Mapping browser requests to specific server-side endpoints and understanding the dynamics of the functions they invoke based on the request method received

Future Work

- If we continued development we would add:
  - Add “Attendee” functionality to Events
  - Automatic Splitting of expense costs across team members
  - Automatic Reassignment of expense costs when a member flakes
  - Automatic Notification of “dropped” pack items when a member flakes
  - React based frontend
  - API based backend

Final Reflection

- We learned how to:
  - Navigate the Full-Stack Lifecycle
  - Master Collaborative Git Workflows
  - Analyze Framework Constraints
  - Align Project Scope to leverage Team Skillset
  - Bridge Technical Gaps with new Tools to us

Thank You

- Questions?
- GitHub: https://github.com/CdreKaputt/CSPB-3308-Team-4-Project
- Trello: https://trello.com/b/3vxc9cLW/flake-cspb-3308-team-4-project